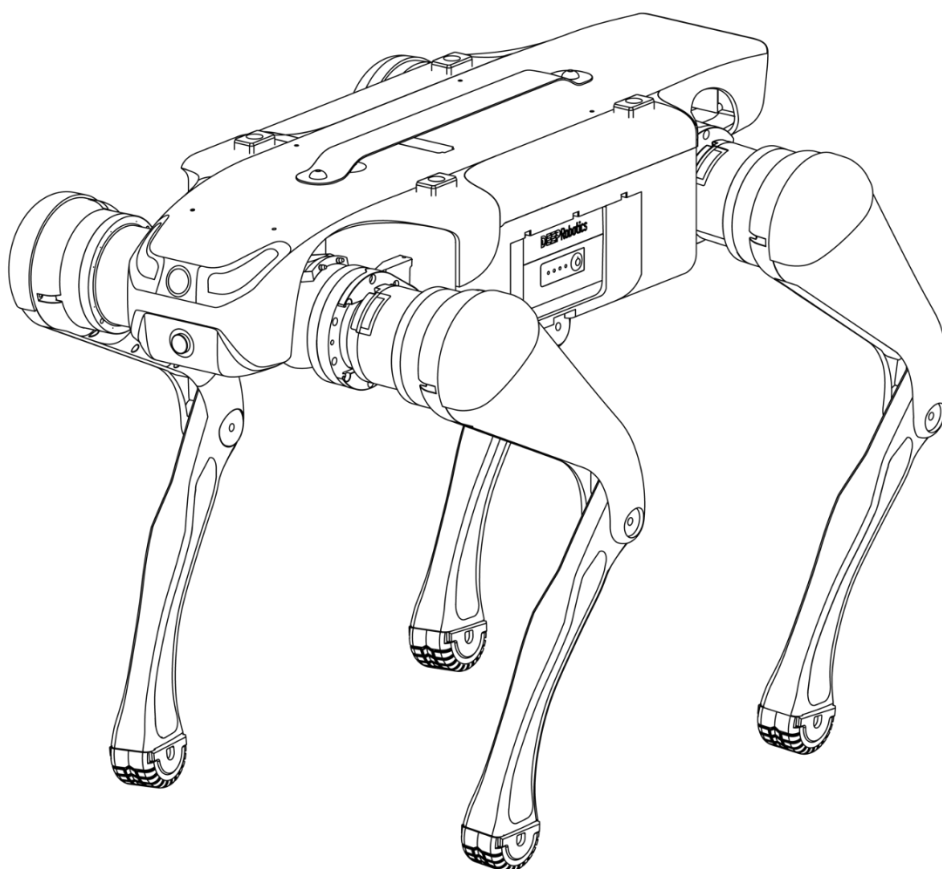


Jueying Lite3 Motion Host Communication Interface (beta)

V1.0.7-0 2024.5.15



Contents

1 UDP Commands	4
1.1 Protocol Format.....	4
1.1.1 Simple Commands.....	4
1.1.2 Complex Commands.....	5
1.1.3 Receiving Complex Commands.....	6
1.2 Control Command Set.....	7
1.2.1 Heartbeat.....	7
1.2.2 Commands for Switching Between Basic States	8
1.2.3 Axis Command.....	9
1.2.4 Commands for Switching Between Motion Modes	10
1.2.5 Commands for Switching Between Different Gaits	10
1.2.6 Action Commands.....	11
1.2.7 Commands for Switching Between Control Modes.....	12
1.2.8 Commands for Saving Data	12
1.2.9 Commands for Keep Stepping Mode	12
1.2.10 Voice Commands and Speaker Commands.....	13
1.2.11 Commands for Perception Settings	14

1.2.12 Speed Commands.....	15
1.3 Command Set for Information Reception.....	15
1.3.1 Robot State	15
1.3.2 Joint Information	20
2 Appendix.....	21
2.1 UDP Message Parsing	21
2.2 Lookup Table for Robot Status	21

Revision History

This document is applicable to motion program jy_exe_87 (20230531).

Version	Updates	Release Date
V1.0.5-0	First Edition	2023/11/20
V1.0.6-0	Delete frontflip	2023/12/13
V1.0.7-0	Modify IP and port (refer to github)	2024/5/15

1 UDP Commands

Communication with the motion host adopts UDP protocol, and the raw data is stored by little-endian.

When developers send messages to the motion host, the target port is 43893 and please refer to [Lite3_MotionSDK](#) section 4.1 to obtain the target IP.

By default, the motion host only reports data to the perception host. If developers need to receive data reported by the motion host, please refer to [Lite3_MotionSDK](#) section 4 and section 5 to connect to the motion host via SSH, modify the data receiving address to the static IP of the development host.

1.1 Protocol Format

This section introduces the raw data format transmitted by UDP protocol. Based on whether complex data is carried, UDP commands can be divided into two categories: simple commands and complex commands.

1.1.1 Simple Commands

The format of a simple command is `[xxxx yyyy zzzz]`, with each letter representing a byte.

The raw data of a simple command is stored in `CommandHead` structure.

```
1 struct CommandHead{
2     uint32_t code;
3     uint32_t paramters_size;
4     uint32_t type;
5 };
```

`xxxx` represents the value of `CommandHead.code`, indicating the command code.

`yyyy` stands for the value of `CommandHead.paramters_size`, indicating the value of the command code. When the command code does not have a valid value, `yyyy = 0`.

`zzzz` refers to the value of `CommandHead.type`, used to differentiate various command type. The type value for simple command is 0, i.e., `zzzz = 0`.

Example of usage:

```
1 // Example Program
2 struct CommandHead command_head = {0};
3 command_head.code = 1;           // Command Code
4 command_head.paramters_size = 0; // Command Value
5 command_head.type = 0;           // Command Type
6 sendto(sfd,&command_head,sizeof(command_head), Target Address, Address Length)
```

1.1.2 Complex Commands

Complex commands carry specific data content in the format of `[xxxx yyyy zzzz bbbbbbbb.....]`, with each letter representing a byte.

The raw data for complex command is stored in `Command` structure.

```
1 struct CommandHead{
2     uint32_t code;
3     uint32_t paramters_size;
4     uint32_t type;
5 };
6 const uint32_t kDataSize = 256;
7 struct Command{
8     CommandHead head;
9     uint32_t data[kDataSize];
10 };
```

`xxxx` represents the value of `Command.head.code`, indicating the command code.

`yyyy` stands for the value of `Command.head.paramters_size`, indicating the length of `bbbbbbbbbb.....`.

`zzzz` refers to the value of `Command.head.type`, used to differentiate various command type. The type value for complex command is 1, i.e., `zzzz = 1`.

`bbbbbbbbbb.....` defines the value of `Command.data`, representing the data carried by the command.

Example of usage:

```
1 // Example Program
2 int32_t to_be_send_data[64];
3 struct Command command = {0};
4 command.head.code = 52; // Command Code
5 command.head.paramters_size = sizeof(to_be_send_data); // Data Length
6 command.type = 1; // Command Type
7 memcpy(&command.data, &to_be_send_data, sizeof(to_be_send_data));
8 sendto(sfd, &command, sizeof(command.head) + command.head.paramters_size, Target Address, Address
Length);
```

1.1.3 Receiving Complex Commands

Taking joint velocity as an example, the usage is as follows:

```
1 struct RobotJointVel{
2     double joint_vel[12];
3 }; // Take joint velocity as an example
4 struct CommandMessage{
5     CommandHead command;
6     uint8_t data_buffer[1024];
7 };
8 CommandMessage cm;
9 size_t recv_size = 0;
10 recv_size = recvfrom(server_fd, &cm, sizeof(cm), 0, (sockaddr*)&client_addr, &sockaddr_in_size);
11 if(cm.command.type == 1){ // Check if it is a complex command
12     if(recv_size == cm.command.paramters_size + sizeof(cm.command)){
13         uint8_t* temp_values = new uint8_t[cm.command.paramters_size];
14         memcpy(temp_values, cm.data_buffer, cm.command.paramters_size);
15         if(cm.command.code == 0x0903){ // Check the command code and parse the data
```

```
16 RobotJointVel *joint_vel = (RobotJointVel *)temp_values;
17     }
18 }
19 }
```

1.2 Control Command Set

Developers control the robot and achieve corresponding functions by directly sending commands to the motion host through UDP protocol. However, sending messages does not change the original control logic.

Jueying Lite3 can receive messages from multiple controllers at the same time. The command of "0x2*****" can be sent by multiple clients, but the downstream data, originating from the wide camera, can only be sent to the client that initially connected to the robot.

1.2.1 Heartbeat

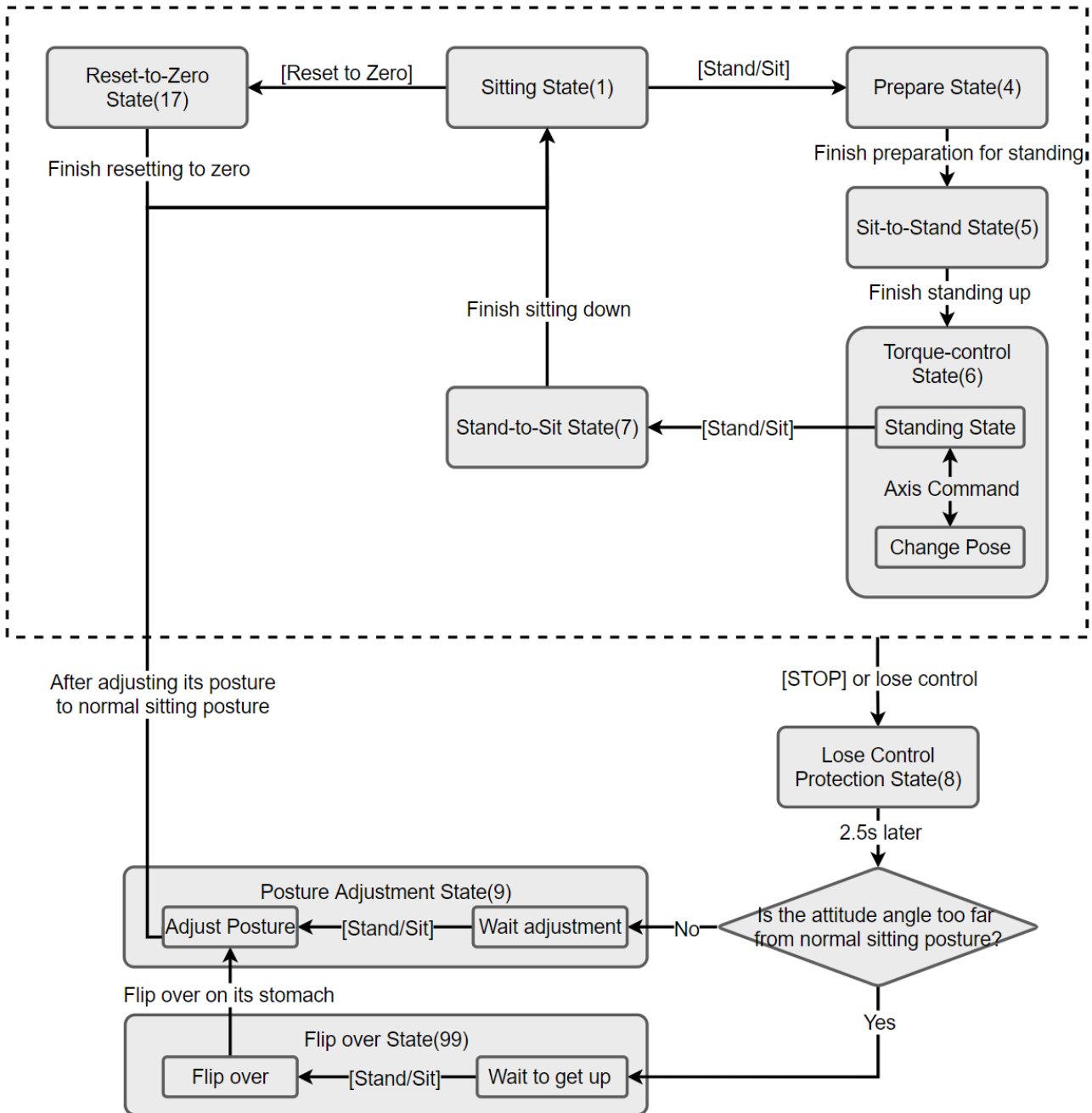
The heartbeat command is used to verify the stability of the connection and should be sent at a minimal frequency of 2Hz.

Command Code	Command Value	Command Type
0x21040001	-	0

[Note] The minus sign - means the command value is meaningless, i.e., the command value will not act on the command.

1.2.2 Commands for Switching Between Basic States

The switch between robot's basic states is achieved by giving corresponding commands, as shown in the following figure (the corresponding code indicated in parentheses is [status value](#)).



Some of the commands are shown in the table below:

Command	Code	Value	Type	Function
Stand/Sit	0x21010202	-	0	Switch between sitting state and standing state
STOP	0x21020C0E	-	0	Enables software-based emergency stop
Reset to Zero	0x21010C05	-	0	Initialize the robot's joints

1.2.3 Axis Command

Axis command is the command output from x-axis and y-axis of the two joysticks of the controller, with a value range of: $[-32767, 32767]$.

1.2.3.1 Axis Command in Pose Mode

In pose mode, the client can change the robot's postures by sending axis command to the motion host. If the motion host doesn't receive any axis command for more than one second, the system will consider the command invalid and the robot will return to its normal standing position. If the axis command value is within the dead zone range, it is considered as 0.

Command	Code	Value	Dead Zone Range	Function
Adjust Roll Angle	0x21010131	0	$[-12553, 12553]$	Positive value rolls to the right
Adjust Pitch Angle	0x21010130	0	$[-6553, 6553]$	Positive value lowers the head
Adjust Body Height	0x21010102	0	$[-20000, 20000]$	Positive value raises the body
Adjust Yaw Angle	0x21010135	0	$[-9553, 9553]$	Positive value rotates to the right

1.2.3.2 Axis Command in Move Mode

In move mode, the client can change robot's moving direction and speed by sending axis commands to the motion host. The issue frequency of axis command should not be lower than 20Hz. If the motion host has not received any axis command for more than 250 ms, the command will be considered as invalid by the system and the robot

will stop moving. When the axis command value is within the dead zone, it is considered as 0, and the robot stops moving. The positive and negative values of the axis command determine the direction of the speed.

Command	Code	Value	Dead Zone	Function
Translation (Left/Right)	0x21010131	0	[-12553,12553]	Specifies the expected linear velocity of the robot along the y-axis, with positive value indicating moving to the right.
Translation (Forward/Backward)	0x21010130	0	[-6553,6553]	Specifies the expected linear velocity of the robot along the x-axis, with positive value indicating moving forward.
Turning Left/Right	0x21010135	0	[-9553,9553]	Specifies the expected angular velocity of the robot, with positive value indicating turning to the right.

1.2.3.3 Stop Command

In move mode, if the axis command is 0, the robot will stop moving.

1.2.4 Commands for Switching Between Motion Modes

Command Name	Command Code	Command Value	Command Type
Pose Mode	0x21010D05	-	0
Move Mode	0x21010D06	-	0

1.2.5 Commands for Switching Between Different Gaits

Robot's gait can be adjusted when it is in move mode.

Command	Code	Value	Type	Function
---------	------	-------	------	----------

Flat gait in Slow gear	0x21010300	-	0	Switch the robot from the current gait to a low-speed gait
Flat gait in Medium gear	0x21010307	-	0	Switch the robot from the current gait to a medium-speed gait
Flat gait in Fast gear	0x21010303	-	0	Switch the robot from the current gait to a high-speed gait
Flat gait in Crawl gear	0x21010406	-	0	Switch the robot from the current gait to a low-speed crawling gait, or switch from a low-speed crawling gait to a normal low-speed gait
RUG gait in Grip gear	0x21010402	-	0	Switch the robot from the current gait to a grip gait
RUG gait in General gear	0x21010401	-	0	Switch the robot from the current gait to a general gait
RUG gait in H-Step gear	0x21010407	-	0	Switch the robot from the current gait to a high-lift stepping gait.

1.2.6 Action Commands

When the robot is in standing or sitting state, action commands can be given to make it perform corresponding action commands.

Command	Code	Value	Type	Conditions for executing actions
Twist	0x21010204	-	0	In Torque-Control state (standing state)
Turn over	0x21010205	-	0	In sitting state
Moonwalk	0x2101030C	-	0	In Torque-Control state (standing state)
Backflip	0x21010502	-	0	In sitting state
Hello	0x21010507	-	0	In sitting state
Long Jump	0x2101050B	-	0	In sitting state
Twist Jump	0x2101020D	-	0	In Torque-Control state (standing state)

1.2.7 Commands for Switching Between Control Modes

The control mode determines the source of speed commands that the robot responds to. In navigation mode, the robot responds to speed commands issued by the perception host, while in manual mode, the robot responds to speed commands issued by the controller.

Command	Code	Value	Type	Function
Navigation	0x21010C03	-	0	Switch the robot from manual mode to navigation mode
Manual	0x21010C02	-	0	Switch the robot from navigation mode to manual mode

1.2.8 Commands for Saving Data

In the event of a malfunction, data saving function allows the robot to save data from the previous 100 seconds, and then the robot will have a soft emergency stop and quit the motion program.

Command Code	Command Value	Command Type
0x21010C01	-	0

1.2.9 Commands for Keep Stepping Mode

When keep stepping mode is enabled, the robot will keep stepping even if no axis commands are received.

Command Code	Command Value	Command Type
0x21010C06	-1=Enable, 2=Disable	0

1.2.10 Voice Commands and Speaker Commands

The App recognizes user voice and sends voice commands with corresponding command values to control robot movement.

Command Code	Command Value	Command Type
0x21010C0A	See the table below	0

The command functions corresponding to the values of the voice commands are shown in the table below:

Value	Meaning
1	Stand up
2	Sit down
3	Move forward
4	Move backward
5	Translate to the left
6	Translate to the right
7	Stop
8	Head down
9	Head up
11	Look left
12	Look right
13	Turn left by 90°
14	Turn right by 90°
15	Turn back by 180°
22	Greeting(perform the action Hello)

Speaker Command:

Command Name	Command Code	Command Value	Command Type
Speaker Command	0x2101030D	0= Turn off the speaker	0
		1= Turn on the speaker	
		2= Check speaker status	

If the speaker command value is set to 2, which means checking the speaker status, the motion host will report the speaker status:

Command Code	Command Type	Content
0x11050f08	0	0=The speaker is off
		1= The speaker is on

1.2.11 Commands for Perception Settings

Developers can send the following commands to the motion host to enable or disable related functions.

Command Name	Command Code	Command Value	Command Type
Turn off all AI options	0x21012109	0x00	0
Enable auto stop function	0x21012109	0x20	0
Enable tracking function	0x21012109	0xC0	0

In addition, developers can send the following commands to the perception host, with the target IP and port as **192.168.1.103:43899**.

Command Name	Command Code	Command Value	Command Type
Enable obstacle avoidance function	0x21012109	0x40	0

If the IP of motion host is 192.168.1.120, the address for streaming the video from robot's wide-angle camera is: **rtsp://192.168.1.120:8554/test**. If the IP of motion host is 192.168.2.1, the address for streaming the video from robot's wide-angle camera is: **rtsp://192.168.2.1:8554/test**.

1.2.12 Speed Commands

The speed commands are complex commands, and the data values each speed command carries are all double-precision floating-point type, which need to be sent to the robot when it is in navigation mode.

Command Name	Command Code	Command Type	Value Range
Angular velocity (rad/s)	0x0141	1	[-1.5,1.5]
Linear velocity in robot x-axis (m/s)	0x0140	1	[-1.0,1.0]
Linear velocity in robot y-axis (m/s)	0x0145	1	[-0.5,0.5]

1.3 Command Set for Information Reception

Users can obtain the information reported by the motion host according to the following command format.

1.3.1 Robot State

Command Code	Command Type	Content	Frequency
0x0901	1	RobotStateUpload structure (details below)	50 Hz

```

1 struct RobotStateUpload{
2     int robot_basic_state;
3     int robot_gait_state;
4     double rpy[3];
5     double rpy_vel[3];
6     double xyz_acc[3];
7     double pos_world[3];
8     double vel_world[3];
9     double vel_body[3];
10    unsigned touch_down_and_stair_trot;
11    bool is_charging;
12    unsigned error_state;
13    int robot_motion_state;
14    double battery_level;
15    int task_state;

```

```

16 |     bool is_robot_need_move;
17 |     bool zero_position_flag;
18 |     double ultrasound[2];
19 | };

```

Field	Type	Meaning
robot_basic_state	int	Robot's current basic state
robot_gait_state	int	Robot's current gait
rpy[3]	double	IMU angle
rpy_vel[3]	double	IMU angular velocity
xyz_acc[3]	double	IMU acceleration
pos_world[3]	double	Robot's posture in world coordinate system
vel_world[3]	double	Robot's velocity in world coordinate system
vel_body[3]	double	Robot velocity in body coordinate system
touch_down_and_stair_trot	unsigned	Invalid data, used as a placeholder only
is_charging	bool	Invalid data, used as a placeholder only
error_state	unsigned	Invalid data, used as a placeholder only
robot_motion_state	int	Robot's motion state
battery_level	double	Battery power percentage in decimal form
task_state	int	Invalid data, used as a placeholder only
is_robot_need_move	bool	Balance state when the robot is subjected to external force
zero_position_flag	bool	Flag indicating the state of Reset-to-Zero
ultrasound[2]	double	Ultrasonic data

The values in the field `robot_basic_state` correspond to the following robot basic states:

Variable Value	Robot's basic state
1	Sitting State
4	Prepare State
5	Sit-to-Stand State
6	Torque-control State
7	Stand-to-Sit State
8	Lose Control Protection State
9	Posture Adjustment State
11	Flipping over state
17	Reset-to-Zero State
18	Backflip State(be performing Backflip action)
20	Hello State(be performing Hello action)

Note: The transition relations for basic states can be referred to in [1.2.2](#).

The values of the field `robot_gait_state` correspond to the following robot gaits:

Variable Value	Gait
0	Flat gait in Slow gear
2	RUG gait in General gear
4	Flat gait in Medium gear
5	Flat gait in Fast gear
6	RUG gait in Grip gear
13	RUG gait in H-Step gear
12	Moonwalk

The field `rpy[3]` may contain the following information:

```
1 | double rpy[3] = {roll,pitch,yaw};
```

Element	Meaning
roll	Roll angle (°) of IMU in world coordinate system
pitch	Pitch angle (°) of IMU in world coordinate system
yaw	Yaw angle (°) of IMU in world coordinate system

The field `rpm_vel[3]` may contain the following information:

```
1 | double rpm_vel[3] = {roll_vel, pitch_vel, yaw_vel};
```

Element	Meaning
roll_vel	Roll angular velocity (rad/s) of IMU in world coordinate system
pitch_vel	Pitch angular velocity (rad/s) of IMU in world coordinate system
yaw_vel	Yaw angular velocity (rad/s) of IMU in world coordinate system

The field `xyz_acc[3]` may contain the following information:

```
1 | double xyz_acc[3] = { x_acc, y_acc, z_acc };
```

Element	Meaning
x_acc	Acceleration (m/s ²) of IMU on x-axis in world coordinate system
y_acc	Acceleration (m/s ²) of IMU on y-axis in world coordinate system
z_acc	Acceleration (m/s ²) of IMU on z-axis in world coordinate system

The field `pos_world[3]` may contain the following information:

```
1 | double pos_world[3] = {x, y, yaw};
```

Element	Meaning
x	X-coordinate value (m) of the robot in world coordinate system
y	Y-coordinate value (m) of the robot in world coordinate system
yaw	Yaw angle (rad) of the robot in world coordinate system

The field `vel_world[3]` may contain the following information:

```
1 | double vel_world[3] = {x_vel, y_vel, yaw_vel};
```

Name	Meaning
x_vel	Linear velocity (m/s) on the x-axis of the world coordinate system
y_vel	Linear velocity (m/s) on the y-axis of the world coordinate system
yaw_vel	Yaw angular velocity (rad/s) in the world coordinate system

The field `vel_body[3]` may contain the following information:

```
1 | double vel_body[3] = {x_vel, y_vel, yaw_vel};
```

Name	Meaning
x_vel	Linear velocity (m/s) on the x-axis of the body coordinate system
y_vel	Linear velocity (m/s) on the y-axis of the body coordinate system
yaw_vel	Yaw angular velocity (rad/s) in the body coordinate system

The meanings of the values in the field `robot_motion_state` are as follows:

Variable Value	Robot's motion state
0	In the state corresponding to the value of <code>robot_basic_state</code>
1	Stepping with the gait corresponding to the value of <code>robot_gait_state</code>
2	The robot is performing Twist
4	The robot is performing Twist Jump
11	The robot is performing Long Jump

Note: The robot's basic status `robot_basic_state`, gait state `robot_gait_state`, and motion state `robot_motion_state` together define the robot's status. Please refer to Appendix 2.2 for details.

The values in the field `is_robot_need_move` have the following meanings:

Variable Value	Balanced state
0	Able to maintain balance
1	Unable to maintain balance, and needs to step to adjust its posture

The values in the field `zero_position_flag` have the following meanings:

Variable Value	Zero position
0	Reset-to-Zero uncompleted or exited
1	Reset-to-Zero completed

The field `ultrasound[2]` may contain the following information:

```
1 | double ultrasound[2] = {forward_distance,backward_distance};
```

Element	Meaning
forward_distance	Distance between the robot and the front obstacle (m)
backward_distance	Distance between the robot and the rear obstacle (m)

Note: The valid range of ultrasound is [0.28m, 4.50m]. When the distance to the obstacle is less than 0.28m, it is displayed as 0.28m. When the distance to the obstacle is greater than 4.50m, it is displayed as 4.50m.

1.3.2 Joint Information

Information of robot joints includes angles, angular velocities and other information.

The command codes are as follows:

Command	Code	Type	Content	Frequency
Angle	0x0902	1	<code>RobotJointAngle</code> structure (details below)	100Hz
Angle velocity	0x0903	1	<code>RobotJointVel</code> structure (details below)	100Hz

```

1  /// Robot's joint angle 0x0902
2  struct RobotJointAngle{
3  double joint_angle[12]; /// Unit:rad
4  };
5  /// Robot's joint angular velocity 0x0903
6  struct RobotJointVel{
7  double joint_vel[12]; /// Unit:rad/s
8  };

```

Each leg of Jueying Lite3 consists of 3 joints: HipX (the hip joint for abduction and adduction), HipY (the hip joint for flexion and extension), and Knee.

The order of the 12 joints is FL_HipX, FL_HipY, FL_Knee, FR_HipX, FR_HipY, FR_Knee, HL_HipX, HL_HipY, HL_Knee, HR_HipX, HR_HipY, HR_Knee (F, H, L and R means front, hind, left and right respectively).

2 Appendix

2.1 UDP Message Parsing

In this section, the joint angle data uploaded by the robot is analyzed as an example.

```

1  /* Raw data, total length of 108 bytes
2   * 0209 0000 6000 0000 0100 0000 2efd 3ccf
3   * 47e4 e1bf f130 4992 f49e e7bf c3d7 eeef
4   * 8567 f13f 9ae4 b66d 6583 f1bf 24d8 f33c
5   * 3d00 f13f 8c16 222a 0b1f 0140 e8ba aaaa
6   * 3803 ef3f d009 0000 60bd d2bf db04 3353
7   * 1150 e73f 7d0b 0000 72f0 e53f 0a3d 0cc3
8   * f071 e73f c3d7 eeef 7b52 f93f
9  */
10 /******
11 ///Content analysis
12 // 0209 0000    // xxxx command code is 0x0902
13 // 6000 0000    // yyyy data length of the main text is 0x60, which is 96 bytes in length
14 // 0100 0000    // zzzz is 1, indicating it is a complex command followed by main text
15 /*
16  * bbbbbbbbbb.....
17  * The following is the main text, with a total of 96 bytes; the data is an array of double
18 type with a length of 12
19  * 2efd 3ccf 47e4 e1bf f130 4992 f49e e7bf
20  * c3d7 eeef 8567 f13f 9ae4 b66d 6583 f1bf
21  * 24d8 f33c 3d00 f13f 8c16 222a 0b1f 0140
22  * e8ba aaaa 3803 ef3f d009 0000 60bd d2bf
23  * db04 3353 1150 e73f 7d0b 0000 72f0 e53f
24  * 0a3d 0cc3 f071 e73f c3d7 eeef 7b52 f93f
25 */
26 // Corresponding to robot joint angles
27 double joint_angle[12] = {-0.5591162726995316, -0.7381537301203293, 1.0877742727584383,
28 -1.0945791516990426, 1.0625584011991203, 2.1401580135014395,
29 0.9691432317102597, -0.2928085327149832, 0.7285238862024487,
30 0.6856012344363617, 0.7326587495353192, 1.5826377828902742};

```

2.2 Lookup Table for Robot Status

The values of (robot_basic_state, robot_gait_state, robot_motion_state)

correspond to the following states in the table:

Value	State
(1,0,0)	Sitting State
(1,0,11)	The robot is performing Long Jump
(4,0,0)	Prepare State
(5,0,0)	Sit-to-Stand State
(6,0,0)	In Torque-control state (standing) with Flat gait in Slow gear
(6,0,1)	Stepping with Flat gait in Slow gear or twisting the body according to axis command
(6,0,2)	The robot is performing Twist
(6,0,4)	The robot is performing Twist Jump
(6,2,0)	In Torque-control state (standing) with RUG gait in General gear
(6,2,1)	Stepping with RUG gait in General gear
(6,4,0)	In Torque-control state (standing) with Flat gait in Medium gear
(6,4,1)	Stepping with Flat gait in Medium gear
(6,5,0)	In Torque-control state (standing) with Flat gait in Fast gear
(6,5,1)	Stepping with Flat gait in Fast gear
(6,6,0)	In Torque-control state (standing) with RUG gait in Grip gear
(6,6,1)	Stepping with RUG gait in Grip gear
(6,12,1)	The robot is performing Moonwalk
(6,13,0)	In Torque-control state (standing) with RUG gait in H-Step gear
(6,13,1)	Stepping with RUG gait in H-Step gear
(7,0,0)	Sitting State
(8,0,0)	Lose Control Protection State
(9,0,0)	Posture Adjustment State
(11,0,0)	Flipping over state
(17,0,0)	Reset-to-Zero State
(18,0,0)	Backflip State(be performing Backflip action)
(20,0,0)	Hello State(be performing Hello action)